

Stateflow Onramp — Complete Notes

Course: Stateflow Onramp ([simulinkR2026a](#) , English) · ~1.5 hours **Source:**

<https://matlabacademy.mathworks.com/details/stateflow-onramp/stateflow> **Prepared for:** HACKSIMUL8 2026 — PES University, 2 Sept 2026

Module and lesson names below are the **actual course structure** read from the live R2026a course page. Technical content is grounded in MathWorks documentation.

Course map (12 modules)

#	Module	Time	Lessons
1	Course Overview	5 min	Course Overview
2	State Machines and Stateflow	5 min	State Machines · Running a Stateflow Chart
3	Creating State Charts	20 min	The Stateflow Editor · Transitions, Conditions, and States · Temporal Logic · Default Transitions and Unreachable States
4	Stateflow Symbols and Data	10 min	Stateflow Data · The Symbols pane · Inputs, Outputs, and Simulink
5	Chart Actions	10 min	Chart Actions · Creating Chart Actions · The During Action
6	Chart Execution	5 min	Chart Execution in Simulink · Summary
7	Project — Robotic Vacuum	5 min	—
8	Flow Charts	10 min	Flow Charts
9	Functions in Stateflow	10 min	Functions in Stateflow · Graphical Functions · MATLAB Functions in Stateflow
10	Chart Hierarchy	5 min	Chart Hierarchy
11	Project — Robotic Vacuum Driving Modes	10 min	—
12	Conclusion	5 min	Additional Resources · Survey

Modules 1–2 — State Machines and Stateflow

What a state machine is

A system that is in **exactly one** of a finite set of **states** at any time, and moves between them along **transitions** when conditions are met.

Stateflow is for **supervisory control and complex decision logic** — the layer that decides *which* mode the system should be in, sitting above the continuous control that decides *how hard to push*.

When to reach for Stateflow instead of Simulink blocks

Use Stateflow when the answer to any of these is yes:

- Does the system have distinct **modes** (Idle / Startup / Running / Fault / Shutdown)?
- Does the correct output depend on **what happened before**, not just current inputs?
- Would a block-diagram version need a tangle of Switch blocks and Unit Delays?
- Do you need **timed** behaviour — "stay here for 5 seconds, then move on"?

Stay in plain Simulink when the logic is a pure function of current inputs. A single Switch block does not need a chart.

Chart anatomy

- **State** — a rounded rectangle, named. Can contain actions and other states.
- **Transition** — an arrow from one state to another, carrying a label.
- **Default transition** — an arrow with a solid dot at its tail and no source state. It marks which state is entered when the chart first activates. **Every chart needs one**, as does every level of hierarchy with parallel decomposition.
- **Junction** — a small circle, used to branch in flow charts.

Transition label syntax

```
event[condition]{condition_action}/transition_action
```

Every part is optional, which is why real labels vary so much:

Label	Means
<code>[x > 5]</code>	condition only — take the transition when <code>x > 5</code>
<code>SwitchOn</code>	event only — take it when event <code>SwitchOn</code> broadcasts
<code>SwitchOn[x > 5]</code>	event and condition must both hold
<code>[x > 5]{y = 0;}</code>	condition action — runs when the condition is evaluated true, <i>before</i> the destination is committed
<code>[x > 5]/{y = 0;}</code>	transition action — runs only once the transition is actually taken

Note the difference between **condition actions** (`{ }`) and **transition actions** (`/ { }`). Condition actions fire during evaluation, even along a path that is later abandoned. Transition actions fire only on a committed transition. When in doubt, use transition actions.

Module 3 — Creating State Charts

The Stateflow Editor

- Add a **Chart** block to a Simulink model from the Stateflow library, then double-click.
- Draw a state: click the state tool, drag a box, type its name.
- Draw a transition: hover a state's edge until crosshairs appear, then drag to the target.
- Click a transition and type to add its label.
- The **Symbols pane** (right-hand side) is where data and events live — see Module 4.

Transitions, conditions and states

Transition evaluation order matters. When several transitions leave the same state, Stateflow evaluates them in a defined order and takes the **first** one that is valid.

- By default the order is determined by geometry (roughly clockwise from twelve o'clock).
- Right-click a state → **Execution Order** to see and set it explicitly.
- **Make the ordering explicit whenever two conditions can be true at once.** Relying on implicit geometric order is how charts break after you drag a box.

Guard your conditions so they are mutually exclusive. `[x > 5]` and `[x > 10]` leaving the same state is a bug waiting to happen; write `[x > 10]` and `[x > 5 && x <= 10]`.

Temporal logic

This is Stateflow's superpower — timed behaviour without any counter plumbing. From the MathWorks reference:

Operator	Meaning
<code>after(n, E)</code>	true once event <code>E</code> has occurred at least <code>n</code> times since the state activated; resets when the state reactivates
<code>before(n, E)</code>	true if <code>E</code> has occurred fewer than <code>n</code> times since activation (Simulink charts only)
<code>at(n, E)</code>	true when <code>E</code> has occurred exactly <code>n</code> times since activation
<code>every(n, E)</code>	true at every <code>n</code> -th occurrence of <code>E</code> since activation
<code>elapsed(sec)</code>	time elapsed since the state activated; equivalent to <code>temporalCount(sec)</code>
<code>temporalCount(sec msec usec)</code>	elapsed time in the given unit
<code>duration(C)</code> or <code>duration(C, sec)</code>	how long condition <code>C</code> has been continuously true while the state was active (Simulink charts only)

Absolute time versus events. Every operator has an event-based form and an absolute-time form. The absolute-time form uses `sec`, `msec` or `usec`:

```
after(5, sec)           % 5 seconds after this state became active
after(10, tick)         % 10 chart wake-ups after activation
every(2, sec)           % every 2 seconds while this state is active
duration(temp > 30, sec) > 5 % temperature has been above 30 for over 5 seconds
```

`tick` is the implicit event that marks each chart wake-up.

Two notations, different meanings:

- `after(5, sec)[condition]` — *trigger* notation. The transition is considered only when the operator fires, and then only if the condition also holds.
- `[after(5, sec) && condition]` — *conditional* notation. The transition can be taken on any wake-up once the operator's condition is satisfied.

Documented constraints:

- Do not use temporal logic on a transition that has no source state, nor on a default transition.
- In Simulink charts, avoid `at` and `every` for absolute-time logic — use `after`.
- Standalone MATLAB charts do not allow temporal logic on multi-source transition paths.

Typical hackathon uses: debouncing a noisy sensor, enforcing a minimum dwell time in a mode, timing out of a startup sequence, blinking an indicator.

Default transitions and unreachable states

- A **default transition** has no source state and a solid dot at its tail. It defines the entry state for the chart or for any subchart.
- **Missing default transition** is the most common beginner error. The chart compiles but never activates a state, so nothing happens.
- An **unreachable state** has no incoming transition. Stateflow flags these — treat the warning as an error, because it means a mode you designed can never be entered.
- Equally check for **dead-end states**: a state with no way out is usually a bug unless it is a deliberate terminal fault mode.

Module 4 — Stateflow Symbols and Data

Data scopes

Set in the **Symbols pane** or the Model Explorer:

Scope	Meaning
Input	arrives from Simulink; creates an input port on the Chart block
Output	sent to Simulink; creates an output port
Local	internal to the chart; persists between time steps
Constant	fixed value, read-only
Parameter	comes from the MATLAB workspace or a mask
Data Store Memory	shared with the rest of the model

The Symbols pane

- Lists every symbol used in the chart, with its scope and type.
- Symbols that appear in the chart but are not yet defined are flagged with a warning triangle. Resolve them by assigning a scope — this is the intended workflow: type the name in the chart first, then declare it in the pane.
- Set **Type** here too (`double` by default; also `boolean` , `int32` , `single` , enums).

Inputs, outputs and Simulink

- Each Input/Output symbol becomes a port on the Chart block in the parent model.
- The chart's **sample time** is inherited by default. For a supervisory controller, giving the chart a fixed discrete sample time is usually the right choice — it makes the logic deterministic and matches how it would run on real hardware.
- **Events** can also cross the boundary: an input event turns the chart into a triggered block, driven by a rising/falling/either edge or a function call.

A useful pattern: output the mode

Add an output such as `mode` and set it in each state's `entry` action:

```
Idle:
    entry: mode = 1;
Running:
    entry: mode = 2;
Fault:
    entry: mode = 3;
```

Log `mode` and plot it beneath the plant response. A stair-step plot showing exactly when the system changed mode is a genuinely persuasive demo artefact.

Module 5 — Chart Actions

The action types

Action	Abbreviation	When it runs
entry	en	once, when the state becomes active
during	du	every time step the state is active <i>and no transition is taken</i>
exit	ex	once, when the state becomes inactive
on <event>	—	when the named event broadcasts while the state is active
bind	—	binds an event or data to the state

Combine them: `entry, during: y = 1;` runs on entry and on every subsequent step.

Writing them

Click into the state below its name and type:

```
Heating
entry:
    heater = 1;
    t_start = t;
during:
    power = Kp*(setpoint - temp);
exit:
    heater = 0;
```

The `during` action — the one people get wrong

`during` runs on every time step the state is active **but only when no valid transition is taken out of the state**. If a transition fires on a given step, the exit action runs instead of the during action.

Practical consequence: continuous computation that must happen every step belongs in `during`; one-shot setup belongs in `entry`; cleanup belongs in `exit`.

Put your safety-critical resets in `exit`, not `entry` of the next state. If a state can be left toward several destinations, `exit` runs regardless of which one is taken — whereas you would otherwise have to duplicate the reset in each destination's `entry`.

Module 6 — Chart Execution

Execution order within one time step

When a Simulink chart wakes up:

1. If the chart is not yet active, take the **default transition** and run the destination state's `entry` action.

2. If the chart is active, evaluate the **outgoing transitions** of the active state, in execution order.
3. If a valid transition is found: run the source state's **exit** action, then the transition action, then the destination state's **entry** action.
4. If no valid transition is found: run the active state's **during** action.
5. With hierarchy, evaluation goes **outermost first** — a parent state's outer transitions are checked before the child's. This is what makes a parent-level fault transition able to pre-empt everything happening inside.

Inner versus outer transitions

- **Outer transition** — leaves the state boundary, so it causes an exit and re-entry.
- **Inner transition** — starts on the inside edge of a state and targets a substate. It redirects within the parent **without** running the parent's exit and entry actions.

Use an inner transition when you want to change substate but keep the parent's context.

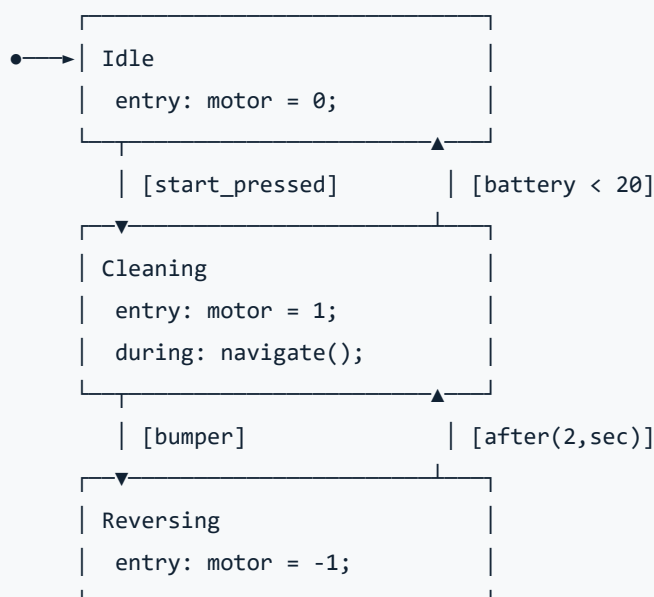
Chart execution in Simulink

- The chart is a block. It executes at its sample time like any other block.
- Inherited sample time is convenient; an explicit discrete rate is more predictable.
- **Super-step semantics** (a chart setting) allows multiple transitions in one time step, so the chart settles fully before returning. Off by default.
- **Debugging**: set breakpoints on states and transitions by right-clicking. During simulation the active state is highlighted, which makes chart debugging far easier than block-diagram debugging. Use this.

Module 7 — Project: Robotic Vacuum

Task: model the supervisory control for a home vacuum robot.

Typical structure:



Concepts it exercises: states as modes, entry actions to command actuators, conditions on sensor inputs, and `after(n, sec)` to give the reverse manoeuvre a fixed duration.

Module 8 — Flow Charts

Flow charts are stateless. They are made of **junctions** (small circles) and transitions, and they run to completion within a single time step. Use them for decision logic that has no memory — the graphical equivalent of `if/elseif/else` or a `for` loop.

- **if/else**: a junction with two outgoing transitions, one guarded `[cond]` and one left unguarded as the default path.
- **for loop**: a junction with a self-loop carrying `{i = i + 1;}` and a guard `[i < N]`.
- Order the outgoing transitions from each junction; the **unguarded** transition must be evaluated **last**, or it will always win.

States versus flow charts, decided in one line: if the logic must remember something between time steps, use states; if it re-decides from scratch every step, use a flow chart.

You can put a flow chart inside a state's action, or draw it standalone in the chart.

Module 9 — Functions in Stateflow

Three kinds, all callable from any state or transition action:

Graphical functions

Flow charts with a signature, drawn inside the chart. Best for logic that is naturally diagrammatic and that you want visible to reviewers.

```
function y = saturate(u, lim)
```

Drawn as a flow chart body, called as `y = saturate(x, 10);`.

MATLAB functions

A block of real MATLAB code inside the chart. Best for mathematics, matrix work, or anything awkward to draw.

```
function d = distance(x1, y1, x2, y2)
    d = sqrt((x2-x1)^2 + (y2-y1)^2);
end
```

Same code-generation constraints as the Simulink MATLAB Function block: variables must be defined before use, and sizes and types must be inferable at compile time.

Simulink functions

Wrap a Simulink subsystem and call it from the chart. Use when the operation needs Simulink blocks — a filter, a lookup table, a transfer function.

Why bother: functions cut duplication and keep charts readable. A chart where the same five lines appear in four states is a chart nobody can review, yours included at 3 pm.

Module 10 — Chart Hierarchy

Superstates and substates

Draw a state inside another state. The outer state is the **superstate** or parent.

- The parent's transitions apply to **all** its children. One transition out of the parent handles a fault from any substate — instead of one arrow per substate.
- Each level with exclusive decomposition needs its **own default transition**.
- Refer to substates with dot notation: `Parent.Child`.
- `in(Parent.Child)` tests whether a particular state is active.

Exclusive (OR) versus parallel (AND) decomposition

- **Exclusive (OR)** — the default. Solid borders. Exactly one substate is active at a time. Use for modes.
- **Parallel (AND)** — dashed borders. **All** substates are active simultaneously, each running its own logic. Use for concurrent concerns.

Right-click the chart or a state → **Decomposition** to switch.

Parallel decomposition is the right tool when your system has independent axes, for example a `MotionMode` state chart and a `BatteryState` chart running side by side. Parallel states execute in a defined order, shown as a number in each state's corner; set it with right-click → **Execution Order**.

Hierarchy is what keeps a chart from becoming spaghetti. If your chart has more than about seven states at one level, it wants grouping.

Module 11 — Project: Robotic Vacuum Driving Modes

Extends the Module 7 project with **hierarchy** — a `Driving` superstate containing the driving modes (`Forward`, `Turning`, `Spiral`, and so on), with a parent-level transition handling the low-battery case for every mode at once.

Concepts exercised: superstates, one transition covering many substates, default transitions at each level, and inner transitions to switch modes without leaving the parent.

Module 12 — Conclusion

Follow-on material: Stateflow documentation, and the wider Simulink learning paths.

Debugging reference

Symptom	Cause	Fix
Chart does nothing, output stays at initial value	no default transition	add one at every level of hierarchy
Chart gets stuck in one state	transition condition never true, or evaluation order wrong	set explicit execution order; check the condition with a Display block
Two transitions both valid, wrong one taken	implicit geometric ordering	right-click → Execution Order, set explicitly
Warning about an unreachable state	no incoming transition	add one, or delete the state
Action runs more often than expected	logic placed in <code>during</code> instead of <code>entry</code>	move one-shot logic to <code>entry</code>
Action does not run on a step	a transition fired, so <code>during</code> was skipped	move the logic to <code>exit</code> , or to the transition action
Chattering between two states	conditions overlap with no hysteresis	separate the thresholds, or add <code>after(n, sec)</code> as a dwell time
Symbol flagged in the Symbols pane	data used but not declared	assign it a scope
Output type error at the Simulink boundary	chart data type does not match the model	set the type explicitly in the Symbols pane

Best debugging habit: run the simulation and watch the chart animate. The active state is highlighted live, so you can see exactly where the logic goes wrong — an advantage block diagrams do not give you.